

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA51 COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.* CO (BL4 , BL5)

Assessment Tool Weightage: 20%

QUESTIONS: 5

TOTAL MARKS: 25

Annexure A CASE STUDY

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Case Study

1. A software company is using MD5 to store user passwords in its database. After a security breach, it is discovered that attackers were able to crack multiple passwords. Analyze the weaknesses of MD5 in this scenario and propose a more secure hashing approach using SHA-based techniques.
2. A banking application implements digital signatures for transaction verification, but users report unauthorized transactions being approved. Analyze the possible flaws in the digital signature implementation and recommend improvements based on secure signature standards.
3. An enterprise network uses Kerberos for authentication, but attackers successfully perform a replay attack to gain unauthorized access. Examine how this attack could occur and propose enhancements in the authentication process to prevent such threats.
4. A secure communication system uses a simple hash function to verify message integrity, but attackers are able to modify messages without detection. Analyze why this approach fails and propose a solution using HMAC to improve message authentication.
5. An organization implements X.509 certificates for secure communication, but users encounter frequent trust validation errors. Analyze the possible issues in certificate management and propose solutions to ensure proper authentication and trust chain validation.
6. A healthcare organization uses RSA encryption to protect sensitive patient records transmitted between hospitals. However, the system experiences performance issues when encrypting large

files. Analyze the limitations of using RSA for large data encryption and propose a hybrid encryption approach using RSA and AES to improve efficiency and security.

7. An e-commerce application uses a symmetric key encryption system to protect customer payment information. After several employees leave the organization, the company discovers that former employees may still have access to encryption keys. Analyze the key management problems in the system and propose a secure key generation, distribution, storage, and revocation mechanism.

8. A government organization uses SHA-1 to generate hashes for important digital documents. Security researchers warn that SHA-1 is vulnerable to collision attacks. Analyze how a collision attack could affect document integrity and recommend a stronger SHA-based hashing algorithm with appropriate security justification.

9. A company has implemented Diffie–Hellman key exchange to establish secure communication between two users. An attacker intercepts the communication and establishes separate keys with both users. Analyze how a Man-in-the-Middle attack can occur in Diffie–Hellman key exchange and propose suitable authentication mechanisms to prevent the attack.

10. An online banking system uses a username and password as the only authentication mechanism. Attackers obtain user credentials through phishing and successfully access customer accounts. Analyze the weaknesses of password-only authentication and propose a multi-factor authentication mechanism using cryptographic techniques, OTPs, or digital certificates to improve account security.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning

Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

CASE STUDY – 2

Title

Analysis and Improvement of Digital Signature Implementation in a Banking Application

Case Study

A banking application implements digital signatures for transaction verification, but users report that unauthorized transactions are being approved. The objective is to analyze the possible weaknesses in the digital signature implementation and recommend improvements based on secure digital signature standards.

1. Introduction

Digital signatures are widely used in banking applications to verify the authenticity and integrity of financial transactions. A digital signature allows a system to determine whether a transaction was authorized by the legitimate user and whether the transaction was modified after signing.

A secure digital signature system generally uses:

- A private key for signing.
- A public key for verification.
- A cryptographic hash function.
- Secure signature algorithms and padding schemes.
- Proper key management.
- Certificate-based identity validation.

If unauthorized transactions are being approved, the problem may not necessarily be with the mathematical RSA algorithm itself. It can result from weaknesses in **private-key protection, transaction data binding, signature verification, certificate management, or application logic**.

2. Problem Identification

The major problem is that transactions are being accepted even though the legitimate users did not authorize them.

Possible causes include:

1. Compromised private keys.
2. Weak or incorrect signature verification.

3. Signing incomplete transaction information.
4. Improper implementation of RSA signatures.
5. Weak cryptographic hash algorithms.
6. Lack of transaction-specific authentication.
7. Poor certificate and key management.
8. Replay of previously valid signed transactions.
9. Inadequate protection of signing devices or credentials.
10. Application logic accepting a transaction without correctly verifying the signature.

3. How the Attack Could Occur

Consider a transaction:

Account A → Account B
Amount = ₹50,000

The application should construct a canonical transaction containing all security-critical fields:

Transaction ID
Sender Account
Receiver Account
Amount
Currency
Timestamp
Nonce

A hash is generated from this complete transaction:

$H = \text{SHA-256}(\text{Transaction Data})$

The hash is then signed using the user's private key.

The banking server verifies the signature using the corresponding public key.

If the implementation signs only part of the transaction, an attacker may be able to alter unsigned fields.

For example:

Signed:
Receiver = Account B

Not adequately protected:
Amount = ₹50,000

An attacker might attempt to change the amount to:

Amount = ₹5,00,000

If the amount is not cryptographically bound to the signature, the server could potentially approve the modified transaction.

4. Major Security Weaknesses

A. Private-Key Compromise

The private key is the most important secret in a digital signature system.

If attackers obtain the private key, they can generate signatures that appear legitimate.

Therefore, private keys should never be stored in plaintext or exposed to ordinary application processes unnecessarily.

B. Weak Signature Algorithm

Older or improperly configured RSA signature schemes may provide inadequate security.

Modern implementations should use secure signature schemes such as:

- RSA-PSS
- ECDSA
- EdDSA, where supported by the system requirements

For RSA, **RSA-PSS with SHA-256 or stronger approved hash functions** is preferable to legacy RSA signature padding.

C. Incomplete Transaction Signing

The signature must cover all security-critical transaction fields.

The signed data should include:

User ID
Transaction ID
Source Account

Destination Account
Amount
Currency
Timestamp
Nonce
Transaction Type

This prevents attackers from changing important transaction information without invalidating the signature.

D. Replay Attack

An attacker could capture a valid signed transaction and submit it again.

For example:

Transaction #1001
Transfer ₹10,000
Signature = Valid

If the server does not check whether transaction #1001 has already been processed, the same signed request might be reused.

A unique transaction ID, timestamp, nonce, and server-side replay detection should therefore be used.

E. Weak Hashing

The signature should be generated over a secure cryptographic hash such as SHA-256 or stronger approved algorithms.

Deprecated hash algorithms such as MD5 and SHA-1 should not be used for new security-sensitive signature systems.

F. Poor Key Management

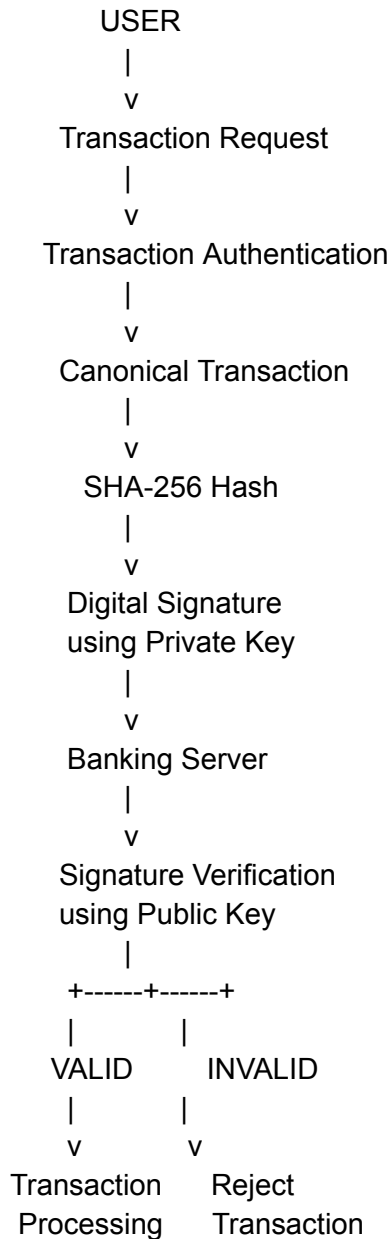
Private keys should be:

- Generated securely.
- Protected using strong access controls.
- Stored securely.
- Rotated according to organizational policy.
- Revoked when compromised.

- Never shared between users.

For high-value banking transactions, hardware-backed key storage such as a **Hardware Security Module (HSM)** can provide stronger protection.

5. Recommended Secure Architecture



6. Recommended Improvements

1. Use Strong Digital Signature Algorithms

Use:

RSA-PSS + SHA-256

or an appropriately configured modern elliptic-curve signature scheme.

2. Protect Private Keys

Private keys should be stored in secure hardware or protected key stores rather than ordinary files.

3. Sign the Complete Transaction

All critical transaction information must be included in the signed message.

4. Implement Replay Protection

Use:

- Unique transaction IDs.
- Nonces.
- Timestamps.
- Transaction expiration.
- Server-side duplicate detection.

5. Verify Certificates

The public key used for verification must be associated with the correct user identity through a trusted certificate or equivalent authenticated key-management system.

6. Use Strong User Authentication

Digital signatures should be combined with strong user authentication and secure transaction authorization mechanisms.

7. Audit Transactions

The banking system should maintain tamper-resistant audit records containing:

Transaction ID

User ID

Time

Amount
Signature status
Verification result
Device/session information

7. Security Comparison

Weak Implementation	Improved Implementation
Weak/legacy signature scheme	RSA-PSS or modern approved signature scheme
MD5/SHA-1	SHA-256 or stronger approved hash
Unprotected private key	Hardware-backed/secure key storage
Partial transaction signing	Complete transaction signing
No replay protection	Nonce + timestamp + transaction ID
Poor certificate validation	Proper certificate/key validation
No auditing	Secure transaction audit logs

8. Expected Outcome

After implementing the recommended security mechanisms:

- Unauthorized transaction modifications should be detected.
- Compromised or invalid signatures should be rejected.
- Replay attempts should be prevented.
- Transaction data should be cryptographically bound to the signature.
- Private keys should be better protected.
- Transaction authentication should become more reliable.

9. Conclusion

The unauthorized transactions could result from weaknesses in the overall digital signature implementation rather than from a fundamental failure of digital signatures. Important vulnerabilities include compromised private keys, incomplete transaction signing, replay attacks, weak cryptographic algorithms, and improper verification.

The banking application should use **RSA-PSS with SHA-256 or another modern secure signature scheme**, protect private keys using secure key-management systems, sign all critical transaction fields, implement nonce and timestamp-based replay protection, validate public-key identities, and maintain secure audit logs.

These measures significantly improve the **integrity, authentication, and non-repudiation** of banking transactions.

CASE STUDY – 7

Title

Secure Cryptographic Key Management for an E-Commerce Payment System

Case Study

An e-commerce application uses a symmetric key encryption system to protect customer payment information. After several employees leave the organization, the company discovers that former employees may still have access to encryption keys.

The objective is to analyze the key-management problems and propose a secure mechanism for **key generation, distribution, storage, rotation, and revocation**.

1. Introduction

Symmetric encryption uses the same secret key for encryption and decryption.

For example:

Plaintext

|

v

Symmetric Encryption

|

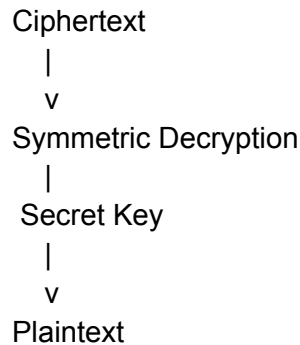
Secret Key

|

v

Ciphertext

To decrypt the information:



The major security challenge is protecting the secret key.

If an unauthorized person obtains the key, encrypted information may be decrypted.

Therefore, effective **cryptographic key management** is essential.

2. Problem Identification

The organization has several key-management weaknesses:

1. Former employees may still possess encryption keys.
2. Keys may be stored in insecure locations.
3. The same key may be shared by multiple employees.
4. There may be no key-rotation policy.
5. Access permissions may not be revoked immediately.
6. Keys may be transmitted through insecure communication channels.
7. There may be no centralized key-management system.
8. Key usage may not be properly monitored or audited.

3. Security Risks

A. Former Employee Access

If an employee leaves the organization but still possesses a valid key, that person could potentially decrypt sensitive information.

B. Shared Keys

If multiple employees use the same symmetric key, it becomes difficult to determine who accessed or misused the key.

C. Key Leakage

Keys stored in:

- Source code
- Text files
- Configuration files
- Email
- Chat messages
- Public repositories

can be accidentally exposed.

D. Lack of Key Rotation

Using the same key for a long period increases the potential impact if the key is compromised.

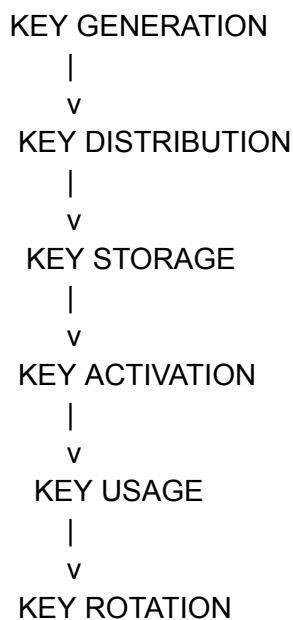
E. Poor Revocation

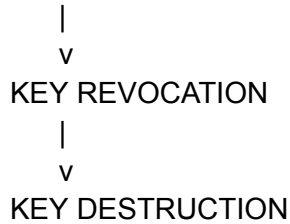
When an employee leaves, their access should be immediately revoked.

If key access remains active, the organization's encrypted information remains at risk.

4. Secure Key Management Lifecycle

A secure key-management system should follow a complete lifecycle:





5. Secure Key Generation

Encryption keys must be generated using a **cryptographically secure random number generator (CSPRNG)**.

Keys should never be generated using predictable values such as:

- Employee ID
- Date of Birth
- Company Name
- Username
- Simple Random Functions

For AES, appropriate key sizes include:

- AES-128
- AES-192
- AES-256

For sensitive payment systems, AES-256 can be selected according to organizational security requirements.

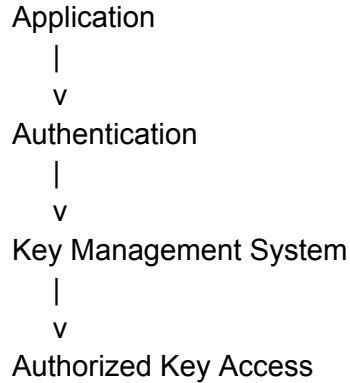
6. Secure Key Distribution

Encryption keys should not be distributed through:

- Email
- SMS
- Plain text files
- Chat applications
- Source code

Instead, keys should be securely provisioned through a centralized key-management system.

For example:



Access should be granted only to authenticated and authorized services or personnel.

7. Secure Key Storage

Secret keys should be stored in a secure **Key Management System (KMS)** or **Hardware Security Module (HSM)** rather than directly in application source code.

The system should provide:

- Encryption at rest.
- Access control.
- Authentication.
- Role-based permissions.
- Key versioning.
- Audit logging.
- Key rotation.
- Revocation.

8. Role-Based Access Control

Employees should receive only the permissions required for their jobs.

Example:

Role	Key Permission
Developer	No direct production key access
Application Service	Required encryption/decryption operation

Security Administrator Key-management administration

Auditor Read audit records

Former Employee No access

This follows the **Principle of Least Privilege**.

9. Key Rotation

Keys should be rotated according to an organizational security policy.

Example:

Key V1 → Active

|

v

Key V2 → New Active Key

|

v

Key V1 → Retired

New transactions should use the current key.

Older keys may need to be retained securely for controlled decryption of historical data, depending on the application's requirements.

10. Key Revocation

When an employee leaves the organization:

Employee Resignation

|

v

Disable Account

|

v

Remove Key Permissions

|

v

Revoke Credentials

|

v

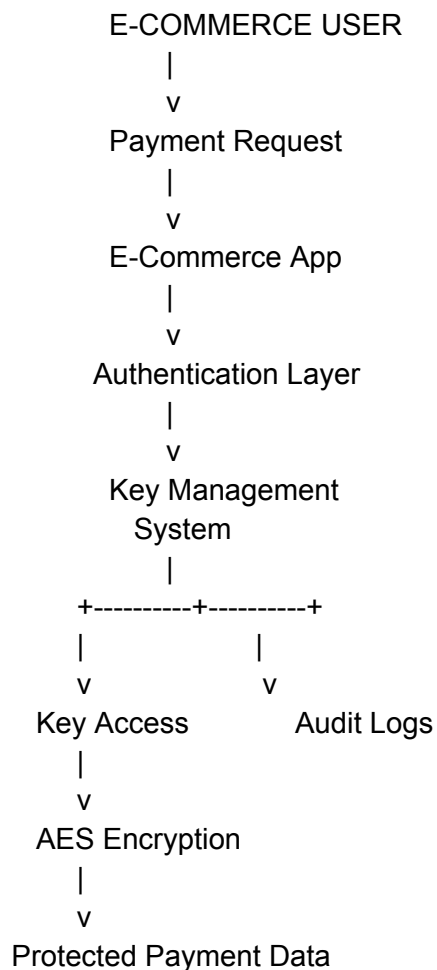
Rotate Potentially Exposed Keys



Audit Key Usage

If there is evidence that the former employee accessed or copied a key, the affected key should be considered compromised and replaced.

11. Secure Key Management Architecture



The application should request cryptographic operations through the approved key-management infrastructure rather than exposing raw secret keys unnecessarily.

12. Recommended Security Measures

1. Centralized Key Management

Implement a centralized KMS or equivalent enterprise key-management infrastructure.

2. Hardware Security

Use HSMs for high-value or highly sensitive cryptographic keys where appropriate.

3. Strong Authentication

Require strong authentication for administrators and privileged users.

4. Role-Based Access Control

Assign key permissions according to job responsibilities.

5. Immediate Employee Offboarding

When an employee leaves:

- Disable accounts.
- Revoke credentials.
- Remove permissions.
- Terminate active sessions.
- Review key-access logs.
- Rotate affected keys when necessary.

6. Key Rotation

Implement a defined key-rotation policy and maintain secure key versions.

7. Audit Logging

Log:

Who accessed the key
Which key was accessed
When it was accessed
What operation was performed
Whether access succeeded or failed

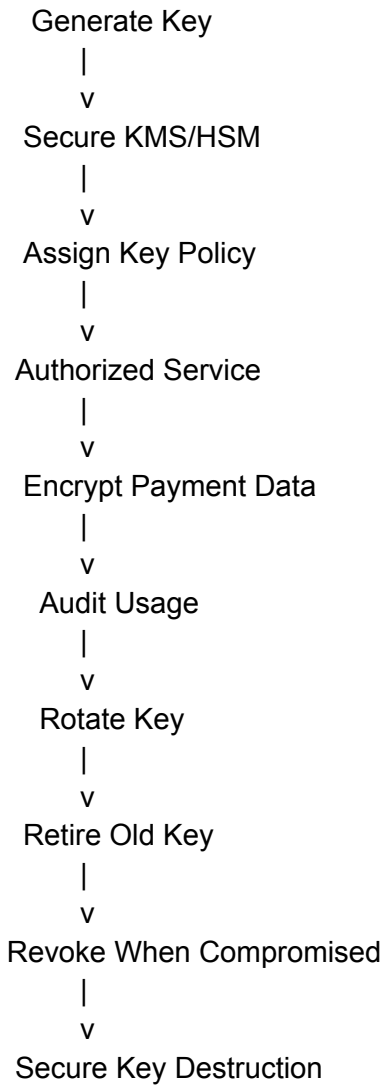
8. Never Hard-Code Keys

Avoid:

```
AES_KEY = "my-secret-key"
```

Keys should be retrieved securely from the approved key-management infrastructure.

13. Example Secure Lifecycle



14. Comparison

Existing System	Recommended System
Shared encryption keys	Controlled key access
Keys may remain with former employees	Immediate access revocation

Keys stored insecurely	KMS/HSM protected storage
No defined rotation	Regular key rotation
Manual distribution	Secure key provisioning
No detailed auditing	Comprehensive audit logs
Broad employee access	Least-privilege access
Long-lived keys	Controlled key lifecycle

15. Expected Outcome

The proposed key-management system will:

- Prevent former employees from accessing active encryption keys.
- Reduce the risk of key theft.
- Provide controlled key distribution.
- Protect keys during storage.
- Enable rapid key revocation.
- Support secure key rotation.
- Improve accountability through audit logs.
- Strengthen protection of customer payment information.

16. Conclusion

The primary weakness in the e-commerce application's security is not necessarily the symmetric encryption algorithm but the **poor management of encryption keys**. Former employees retaining access demonstrates inadequate access control, key revocation, and employee offboarding procedures.

A secure solution should implement a centralized **KMS/HSM-based key-management architecture**, cryptographically secure key generation, protected key storage, role-based access control, secure distribution, regular key rotation, immediate revocation, and comprehensive auditing.

Effective key management ensures that even strong encryption algorithms such as AES remain secure throughout their complete lifecycle and significantly reduces the risk of unauthorized access to sensitive customer payment information.

Overall Case Study Conclusion

Case Study 2 demonstrates the importance of correctly implementing **digital signatures** in banking systems, while Case Study 7 demonstrates the importance of **cryptographic key management** in e-commerce systems.

Digital signatures protect transaction authenticity and integrity, whereas secure key management protects the secret cryptographic keys on which encryption systems depend. Both are essential components of a secure modern information system.